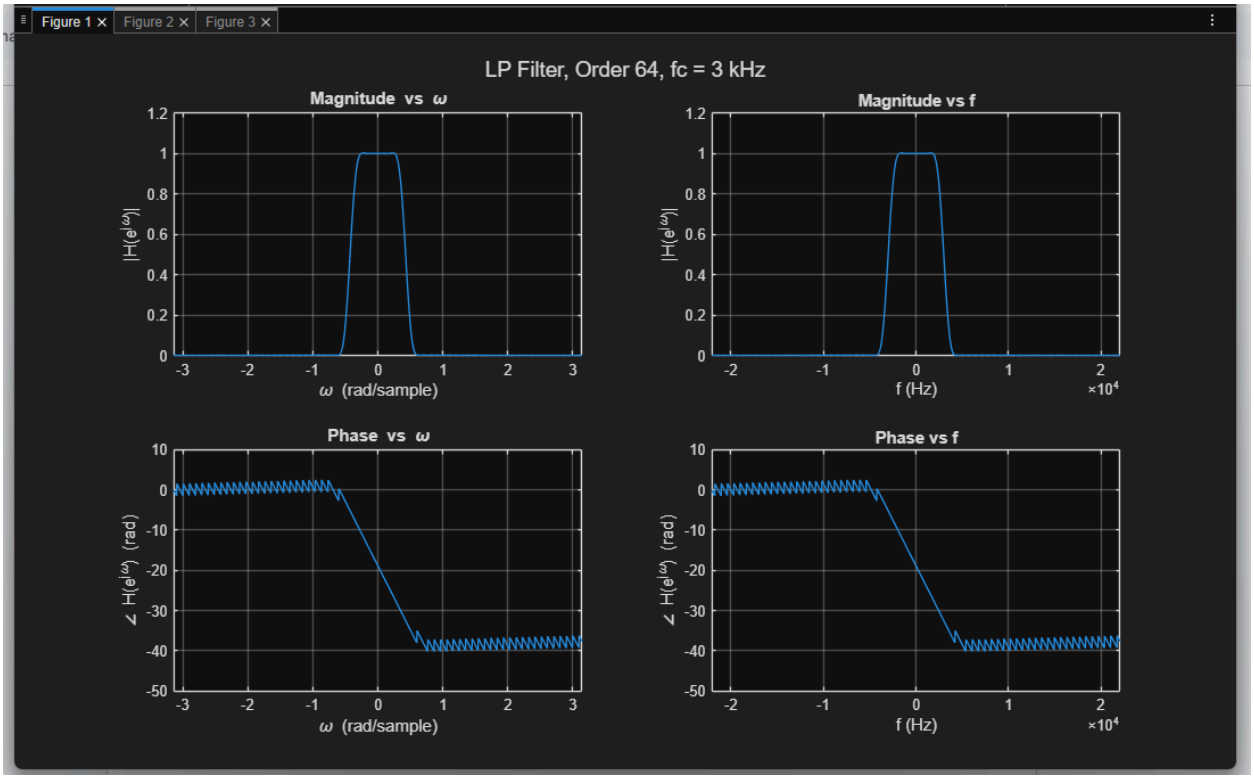
Exercise 0:

For Exercise 1 I used the two example files provided, pirates.wav and papas_megalitriti.wav. For Exercise 2 I recorded 9 short clips on my phone, each 2 to 3 seconds long, and converted them to WAV at 44.1 kHz. The 9 songs were Enter Sandman, Hotel California, Eyes Without a Face, Power, Runaway, Headlines, Cornfield Chase, Stronger, and Pyramids. I also recorded test_a.wav and test_b.wav for the matching part of Exercise 2, which are second recordings of Power and Enter Sandman taken with the phone held a bit further from the speaker so the audio came out slightly different.

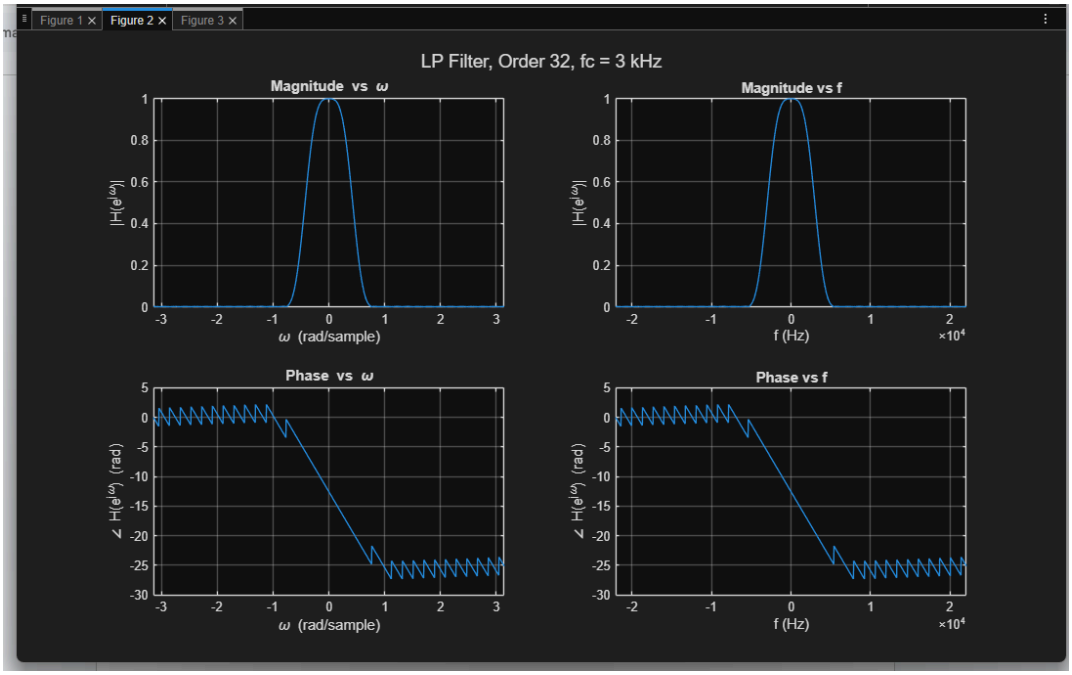
Exercise 1: Fun with FIR filtering

Part 2: LP Order 64



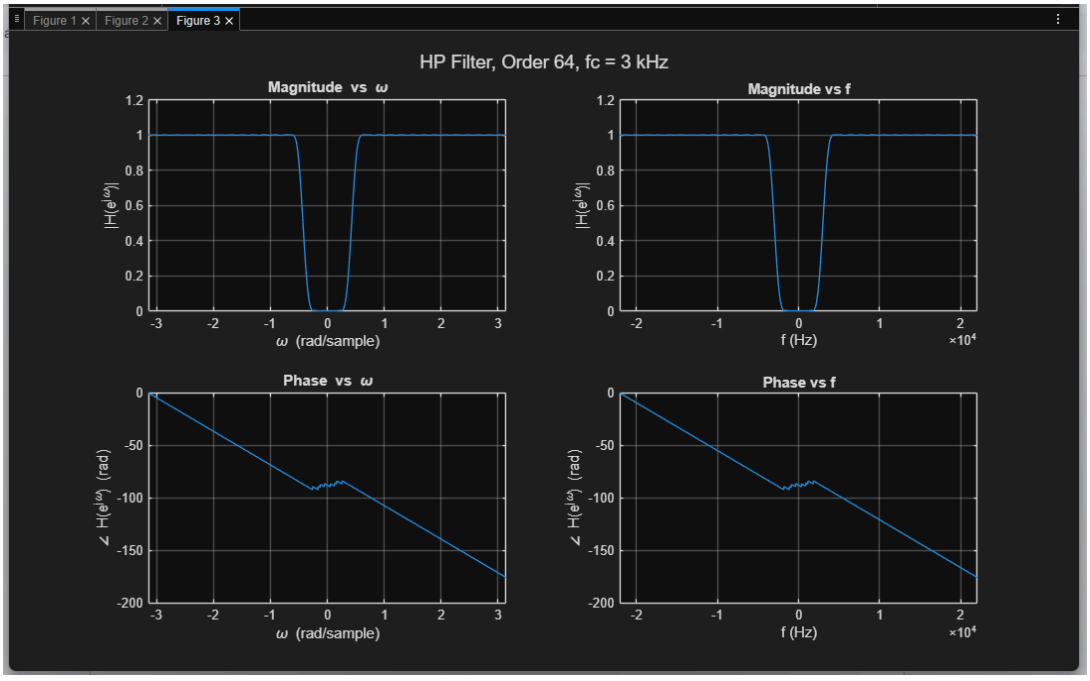
The filtered audio sounded muffled with all the high frequency content gone, so things like the highs of the orchestral instruments in pirates.wav were missing. The magnitude plot shows a clean low pass shape with a sharp transition near 3 kHz, and the phase is linear inside the passband which is expected from a Type I FIR filter with symmetric coefficients and a constant delay of M/2 samples.

Part 3: LP Order 32



Cutting the filter order in half made a clear difference. The passband still reaches 3 kHz but the transition band is noticeably wider, since fewer coefficients give less control over how sharp the cutoff is. The phase slope is also shallower because group delay scales with M/2, so order 32 has half the delay of order 64. The audio still sounded muffled but the cutoff felt less aggressive than the order 64 case. So the trade off is what you would expect, less computation and less delay but also a less precise frequency response.

Part 4: HP Order 64



The high pass filter cut everything below 3 kHz, so the audio became thin with the bass and body gone. The magnitude is the inverse shape of the low pass case. The phase plot looks much cleaner here than in the LP versions because the magnitude is non zero almost everywhere, so the phase is well defined across the whole frequency range. A high pass FIR filter is a low pass filter with all of its zeros shifted by pi radians around the unit circle, which flips the response.

Exercise 2 - Fun with Fingerprinting Identification

Part 1:

I put the project Part B fingerprinting code into a function called compute_fingerprint that takes a filename and returns the hash matrix. The script exercise2_part1.m loops over the 9 songs, calls the function on each, and saves all the hash matrices into database.mat for the matching script to load later. One thing I ran into early on is that the default ampMin = 10 threshold assumes audio with peak amplitude near 1. My phone recordings were quieter, with peaks more like 0.1 to 0.3, so most spectrogram magnitudes never crossed the threshold and I got almost no peaks. For test_b I was getting zero hashes. I added one line to normalize each clip to peak amplitude 1.0 before the spectrogram, and after that the peak detection worked consistently.

Hash Table:

Song 1: Enter Sandman Total hashes generated: 2305 First 10 hashes [f1, f2, deltaT, t1]: 1.0e+03 * 0.4200 1.4970 0 0.0002 0.4200 3.1650 0 0.0002 0.4200 0.6680 0.0001 0.0002 0.4200 2.3260 0.0001 0.0002 0.4200 2.8320 0.0001 0.0002 1.4970 3.1650 0 0.0002 1.4970 0.6680 0.0001 0.0002 1.4970 2.3260 0.0001 0.0002 1.4970 2.8320 0.0001 0.0002 1.4970 3.0040 0.0001 0.0002	**Song 2: Hotel California** Total hashes generated: 1180 First 10 hashes [f1, f2, deltaT, t1]: 1.0e+03 * 0.3660 3.6500 0 0.0002 0.3660 0.8720 0.0001 0.0002 0.3660 1.7440 0.0001 0.0002 0.3660 1.8300 0.0001 0.0002 0.3660 1.9810 0.0001 0.0002 3.6500 0.8720 0.0001 0.0002 3.6500 1.7440 0.0001 0.0002 3.6500 1.8300 0.0001 0.0002 3.6500 1.9810 0.0001 0.0002 3.6500 2.2390 0.0001 0.0002	**Song 3: Eyes Without a Face** Total hashes generated: 1720 First 10 hashes [f1, f2, deltaT, t1]: 1.0e+03 * 0.4950 0.8400 0 0.0002 0.4950 1.1300 0 0.0002 0.4950 1.5070 0 0.0002 0.4950 1.5830 0 0.0002 0.4950 1.0010 0.0001 0.0002 0.8400 1.1300 0 0.0002 0.8400 1.5070 0 0.0002 0.8400 1.5830 0 0.0002 0.8400 1.0010 0.0001 0.0002 0.8400 1.2600 0.0001 0.0002
Song 4: Power Total hashes generated: 1485 First 10 hashes [f1, f2, deltaT, t1]: 1.0e+03 * 1.6470 1.9160 0 0.0002 1.6470 2.9820 0 0.0002 1.6470 4.4570 0 0.0002 1.6470 0.3120 0.0001 0.0002 1.6470 0.6350 0.0001 0.0002 1.9160 2.9820 0 0.0002 1.9160 4.4570 0 0.0002 1.9160 0.3120 0.0001 0.0002 1.9160 0.6350 0.0001 0.0002 1.9160 3.0900 0.0001 0.0002	**Song 5: Runaway** Total hashes generated: 1330 First 10 hashes [f1, f2, deltaT, t1]: 1.0e+03 * 5.0820 1.3240 0.0001 0.0002 5.0820 0.2050 0.0001 0.0002 5.0820 1.2490 0.0001 0.0002 5.0820 1.4530 0.0001 0.0002 5.0820 3.9730 0.0001 0.0002 1.3240 0.2050 0.0001 0.0002 1.3240 1.2490 0.0001 0.0002 1.3240 1.4530 0.0001 0.0002 1.3240 3.9730 0.0001 0.0002 1.3240 0.6570 0.0001 0.0002	**Song 6: Headlines** Total hashes generated: 2360 First 10 hashes [f1, f2, deltaT, t1]: 1.0e+03 * 0.8290 0.4950 0.0001 0.0002 0.8290 1.5720 0.0001 0.0002 0.8290 2.1640 0.0001 0.0002 0.8290 2.2720 0.0001 0.0002 0.8290 2.6810 0.0001 0.0002 0.4950 1.5720 0 0.0002 0.4950 2.1640 0 0.0002 0.4950 2.2720 0 0.0002 0.4950 2.6810 0 0.0002 0.4950 3.4350 0 0.0002
Song 7: Cornfield Chase Total hashes generated: 985	**Song 8: Stronger** Total hashes generated: 1970	**Song 9: Pyramids** Total hashes generated: 880

First 10 hashes [f1, f2, deltaT, t1]: 1.0e+03 * 0.4410 0.9800 0 0.0002 0.4410 1.6580 0 0.0002 0.4410 1.7660 0 0.0002 0.4410 2.9610 0 0.0002 0.4410 0.3340 0.0001 0.0002 0.9800 1.6580 0 0.0002 0.9800 1.7660 0 0.0002 0.9800 2.9610 0 0.0002 0.9800 0.3340 0.0001 0.0002 0.9800 0.5280 0.0001 0.0002	First 10 hashes [f1, f2, deltaT, t1]: 1.0e+03 * 0.7750 0.9370 0 0.0002 0.7750 0.1940 0.0001 0.0002 0.7750 0.3880 0.0001 0.0002 0.7750 1.0980 0.0001 0.0002 0.7750 2.4980 0.0001 0.0002 0.9370 0.1940 0.0001 0.0002 0.9370 0.3880 0.0001 0.0002 0.9370 1.0980 0.0001 0.0002 0.9370 2.4980 0.0001 0.0002 0.9370 3.1010 0.0001 0.0002	First 10 hashes [f1, f2, deltaT, t1]: 1.0e+03 * 0.7000 1.2380 0 0.0002 0.7000 1.1740 0.0001 0.0002 0.7000 1.3890 0.0001 0.0002 0.7000 1.5610 0.0001 0.0002 0.7000 0.1290 0.0001 0.0002 1.2380 1.1740 0.0001 0.0002 1.2380 1.3890 0.0001 0.0002 1.2380 1.5610 0.0001 0.0002 1.2380 0.1290 0.0001 0.0002 1.2380 0.4090 0.0001 0.0002

Part 2:

Fingerprinting

The fingerprint is essentially a compressed list of which spectral peaks show up in the song and how those peaks line up in time relative to each other. It gets built by computing the spectrogram of the audio, then finding local maxima where a time-frequency cell is larger than every other cell in a small neighborhood around it. The Part B code uses a 7 by 7 box for that neighborhood. After that, each peak gets paired with the next few peaks that come after it in time, fanOut = 5 in the project code, and each pair becomes one hash. So a hash is a small piece of the song which gives a frequency pair and the time gap between them.

I think the main reason this kind of fingerprint handles noise well is that local maxima are stable under small noise, since the loudest cell in a neighborhood usually stays the loudest even after some noise gets added. The hash also uses time gaps rather than absolute times, so where the recording starts in the song does not matter. fanOut = 5 builds in redundancy too because each peak shows up in multiple hashes, so even if some get corrupted the rest can still match. And the ampMin threshold throws out the weakest peaks, which are the ones most likely to be noise rather than real musical content anyway. The limits show up when noise is loud enough to create new spectral peaks that compete with the song peaks, which would generate wrong hashes. Band limited recordings are also a problem. My phone recordings did not capture the full audio spectrum so most of my fingerprints ended up in the midrange, which is just what happens with consumer devices.

Part 3:

Closeness

For closeness, just counting how many hashes overlap is not enough because two unrelated songs can share hashes by coincidence. The best way is to also check whether the matching hashes line up in time consistently. If the query is a clip from a song in the database, then for every matching hash the difference between when it appears in the database and when it appears in the query should be the same number, since the query is just a time shifted copy. Building a histogram of these time offsets should show a sharp peak at one specific offset for the right song, while a wrong song produces spread apart offsets and a flat histogram. So the score for each candidate song is the height of the tallest bar in its time offset histogram. This is the same idea Shazam uses. I implemented this in match_song.m. For each song in the database it finds matching hashes based on the f1, f2, deltaT triple, computes time offsets, and returns the histogram peak as the score. The script exercise2_part3.m runs the matching on test_a.wav and test_b.wav.

```
>> exercise2_part3

============================================================
Matching: test_a.wav (expected: song 4 - Power)
============================================================
Query hash count: 1790

Score per song (higher = better match):
  Song 4 (Power): 58   <-- BEST
  Song 5 (Runaway): 4
  Song 6 (Headlines): 3
  Song 8 (Stronger): 3
  Song 1 (Enter Sandman): 2
  Song 2 (Hotel California): 1
  Song 3 (Eyes Without a Face): 1
  Song 9 (Pyramids): 1
  Song 7 (Cornfield Chase): 0

Predicted: song 4 (Power)
Expected:  song 4 (Power)
Result: CORRECT

============================================================
Matching: test_b.wav (expected: song 1 - Enter Sandman)
============================================================
Query hash count: 3340

Score per song (higher = better match):
  Song 1 (Enter Sandman): 23   <-- BEST
  Song 6 (Headlines): 5
  Song 3 (Eyes Without a Face): 4
  Song 5 (Runaway): 4
  Song 8 (Stronger): 4
  Song 2 (Hotel California): 3
  Song 7 (Cornfield Chase): 3
  Song 4 (Power): 2
  Song 9 (Pyramids): 1

Predicted: song 1 (Enter Sandman)
Expected:  song 1 (Enter Sandman)
Result: CORRECT
```

Test_a correctly matched Power with a score of 58 against a second place of 4, so the margin is 54. Test_b correctly matched Enter Sandman with a score of 23 against a second place of 5, margin 18.

An issue I had along this process is that I had a sample rate mismatch early on where my songs came out at 48 kHz from the converter while the example files were at 44.1 kHz. Since the hashes round frequencies to integers and spectrogram bins depend on sample rate, the same pitch was rounding differently and the matching could not find any overlap. Re-converting everything to 44.1 kHz fixed it.